# Django, paso a paso

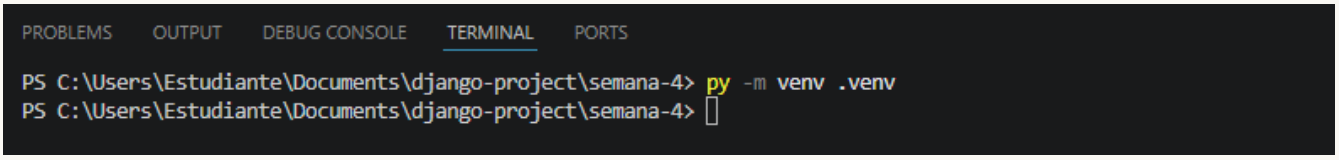
CUADERNO DE PRÁCTICA / SEMANA 04

Del entorno virtual a una vista conectada a la base de datos. Dieciocho pasos para repetir la práctica, con comandos listos para consultar y capturas del proceso.

## 1. Crear el entorno virtual

```
py -m venv .venv
```

En la carpeta del proyecto, crea **.venv** para separar sus dependencias.




## 2. Elegir el intérprete

**Ctrl + Shift + P** → **Python: Select Interpreter** → elige el Python de **.venv**. Seleccionarlo en el editor y activar el entorno en la terminal son acciones distintas.

## 3. Instalar Django

```
py -m pip install "Django>=5.2,<5.3"
```

Instala Django **5.2.x**. Hazlo con el entorno activo; en capturas posteriores se ve **(.venv)**.

## 4. Comprobar la versión

```
py -m django --version
```

Debe mostrar **5.2.x**. En la captura del paso 18 se confirma la versión **5.2.17**.

# Crear el proyecto y su app

## 5. Crear el proyecto

```
django-admin startproject backend_store .
```

**backend_store** es el nombre; el **punto** indica la carpeta actual. Genera **manage.py** y la carpeta de configuración **backend_store**.

**manage.py** sirve para ejecutar tareas del proyecto, como iniciar el servidor. Django ya se instaló con pip.

## 6. Crear la aplicación de productos

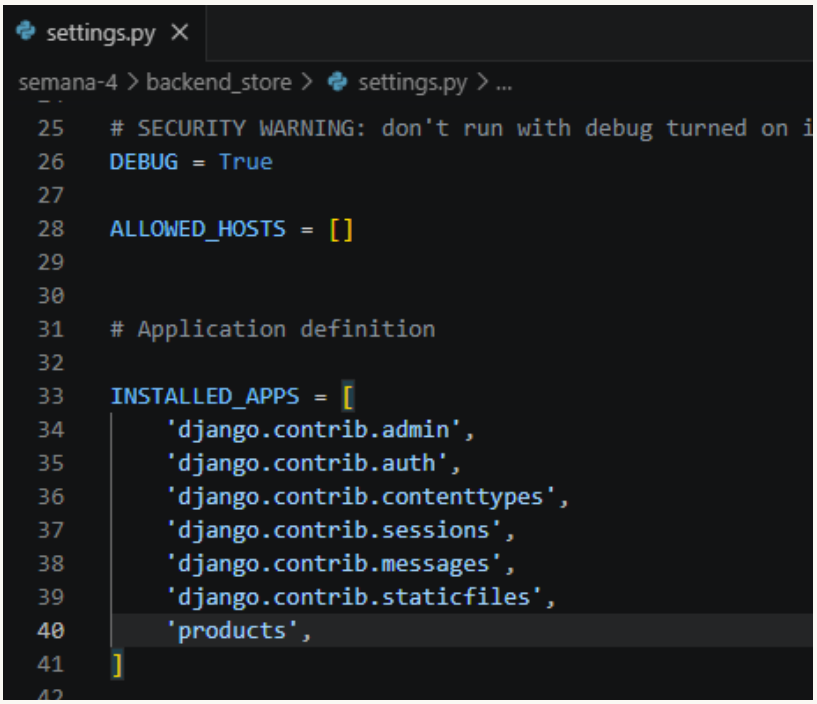
```
py manage.py startapp products
```

Ejecuta desde donde está **manage.py**. Crea **products** con archivos como **models.py** (datos), **views.py** (respuestas) y **admin.py** (administración).

## 7. Registrar la app

En **backend_store/settings.py**, agrega esta línea dentro de **INSTALLED_APPS** y guarda:

```
'products',
```




**Recuerda:** registra aquí cada nueva app Django que vayas a utilizar.

# La primera respuesta

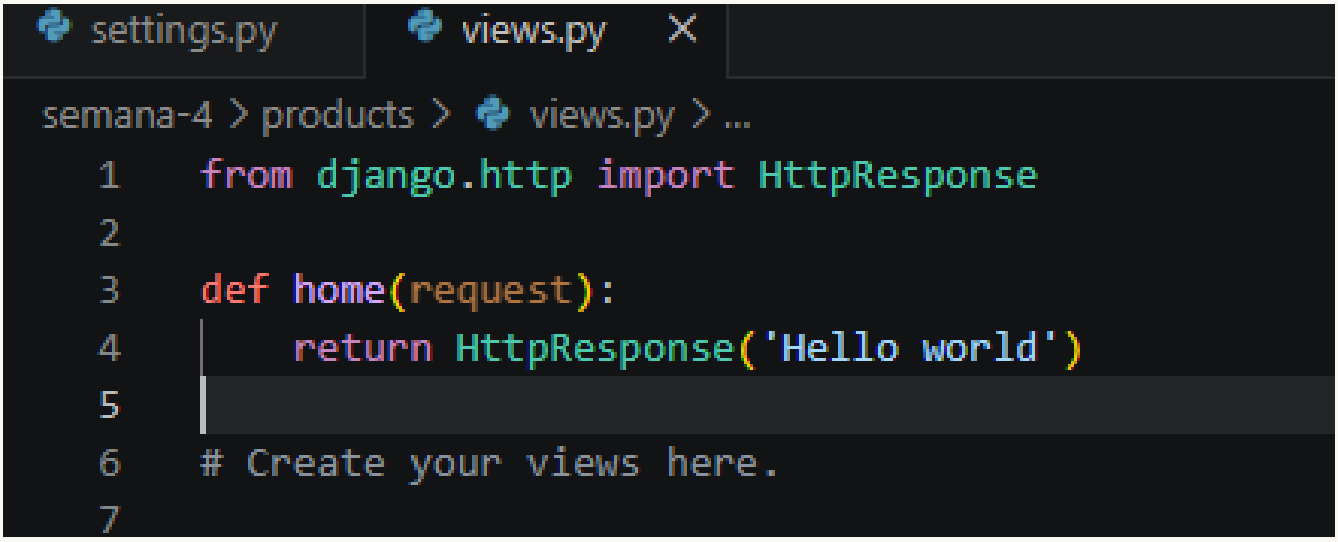
## 8. Crear la vista home

Abre **products/views.py** y escribe:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse('Hello world')
```

**request** recibe la petición; **HttpResponse** devuelve el texto **Hello world**.

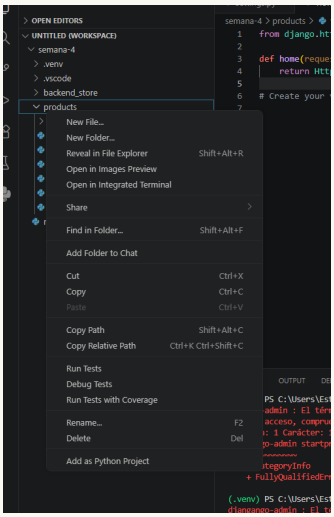


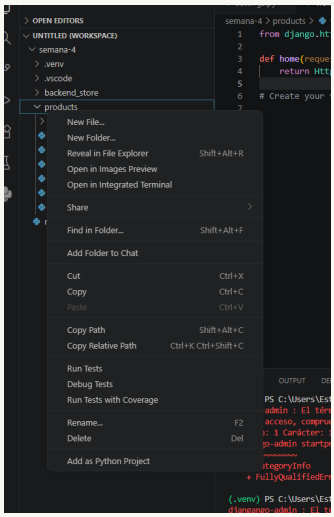

**Idea clave:** cada app puede tener sus propias vistas. Para visitarlas desde el navegador, debes conectarlas a rutas.

# Las rutas de products

## 9. Crear el archivo de rutas

Clic derecho en **products** → **New File**. En tu captura lo llamaste **url.py**; para seguir el código del paso 10, usa **urls.py**.

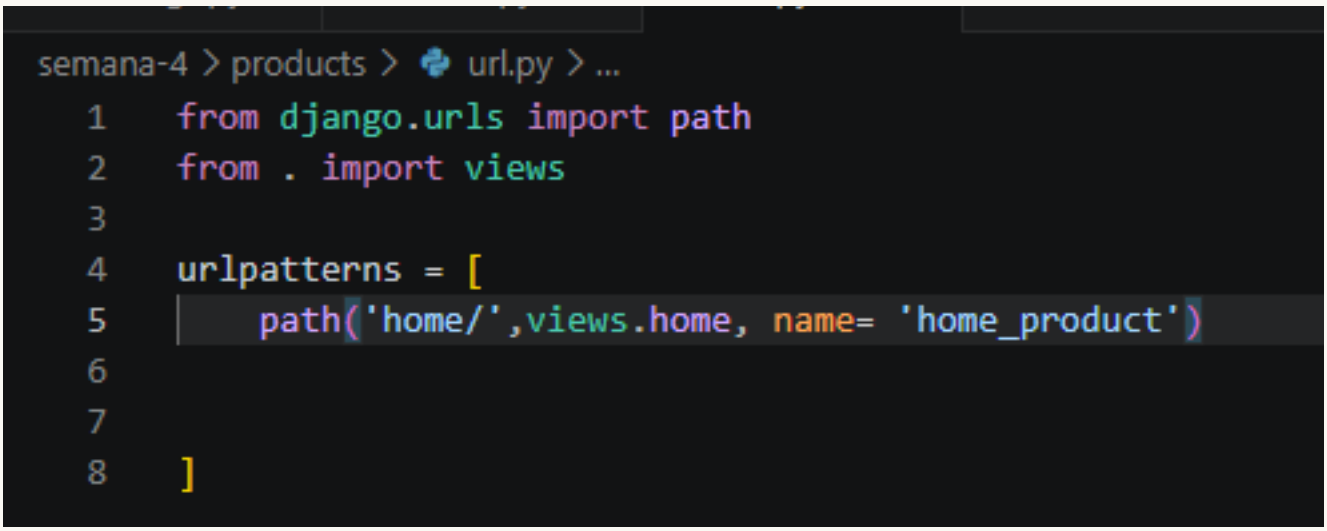



```
from django.urls import path
from . import views

urlpatterns = [
    path('home/', views.home, name='home_product'),
]
```

**home/** es la ruta; **views.home** es la vista; **home_product** es el nombre de la ruta para usarla desde código o plantillas, no el de un HTML.




**Recuerda:** startapp no genera este archivo. Cada endpoint que expongas necesita una ruta.

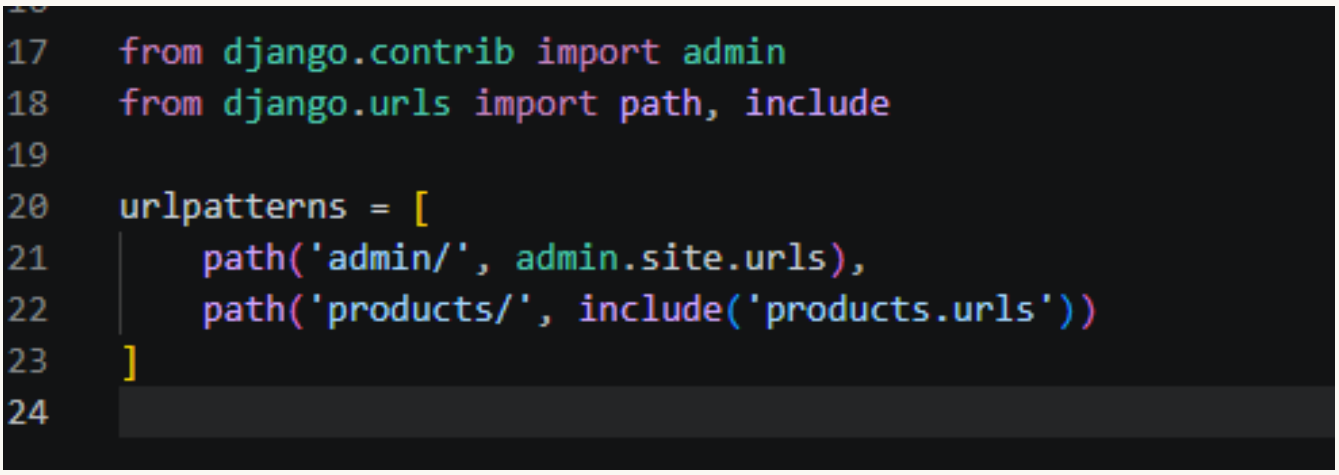
# Conectar la app al proyecto

## 10. Incluir las rutas de products

En **backend_store/urls.py**, importa **include** y agrega el prefijo **products/**:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('products/', include('products.urls')),
]
```




**El nombre debe coincidir: products.urls** requiere **products/urls.py**. Si conservas **url.py**, usa **products.url**.

**Así se forma la dirección:** products/ + home/ = **/products/home/**.

Django ejecuta la vista cuando recibe una petición a esa ruta. Las vistas no se ejecutan todas al encender el servidor.

# Probar que funciona

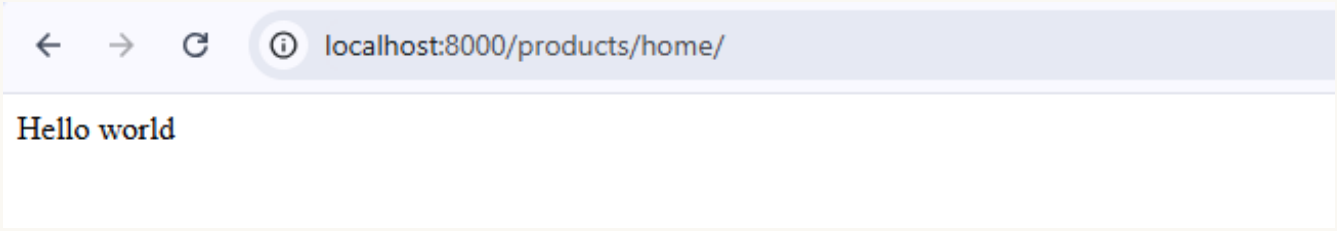
## 11. Iniciar el servidor y abrir la ruta

Desde la carpeta donde está **manage.py**:

```
py manage.py runserver
```

Deja la terminal abierta y visita:

```
http://localhost:8000/products/home/
```




**Prueba superada:** tu captura confirma que el servidor responde y que la ruta llega a la vista **home**.

**Detener el servidor:** pulsa **Ctrl + C** en la terminal.

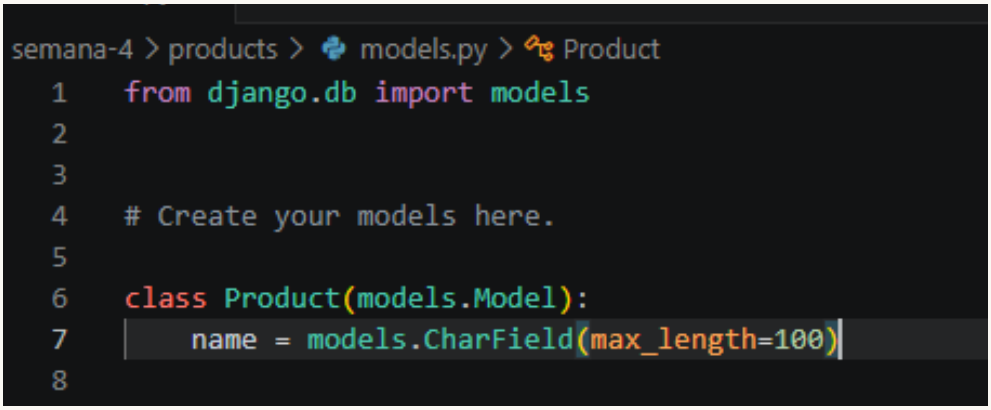
**Lo que sigue:** agregar módulos y funcionalidades. Para cada nueva app: crearla, registrarla y conectar sus rutas si tiene endpoints.

Para repasar: app → vista → ruta de la app → include del proyecto → navegador.

# Definir los datos del producto

## 12. Crear el modelo Product

Abre el **archivo products/models.py**. **Product(models.Model)** hereda de la clase base de modelos de Django; dentro se definen sus atributos.



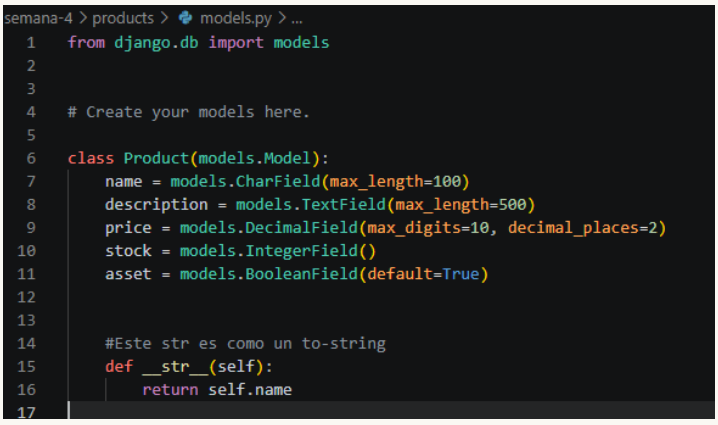

**CharField(max_length=100)**: texto corto, como un nombre, con un máximo de 100 caracteres.

```
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=100)
    description = models.TextField(max_length=500)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.IntegerField()
    asset = models.BooleanField(default=True)

    def __str__(self):
        return self.name
```

**TextField**: almacena textos extensos, como descripciones. En formularios se muestra como un área de texto de varias líneas. Aquí **max_length=500** se refleja en el formulario; no impone por sí solo ese límite en el modelo ni en la base de datos.




Fuente: documentación oficial de Django 5.2 · TextField.

# Del modelo a la base de datos

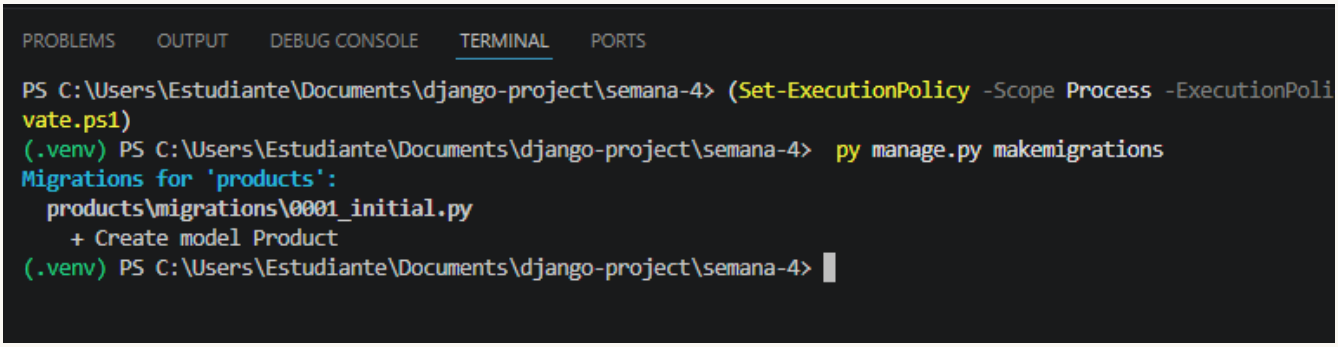
## 13. Crear y aplicar migraciones

**¿Qué son?** Archivos que describen y versionan cambios en los modelos para aplicarlos a la base de datos. En este caso, permiten crear la tabla de **Product**.

### 1. Crear la migración

```
py manage.py makemigrations
```

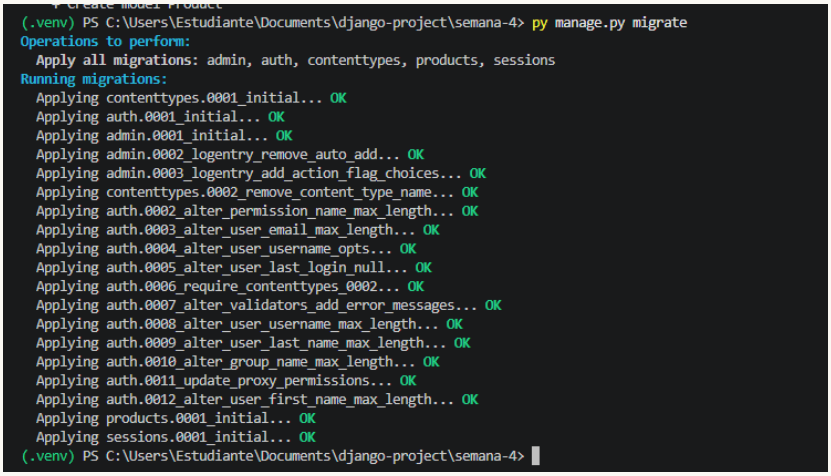
Genera **products/migrations/0001_initial.py** con la descripción de la creación del modelo. La carpeta **migrations** ya venía con la app; este comando aún no crea la tabla.




### 2. Aplicar las migraciones

```
py manage.py migrate
```

Aplica los cambios pendientes: crea la tabla **products_product** y, en esta primera ejecución, las tablas de las apps incluidas de Django.




**Sobre db.sqlite3:** es el archivo de la base de datos y ya aparecía en tus capturas anteriores. Su existencia no demuestra que la tabla Product ya esté creada ni que el archivo esté vacío.

# Revisar las migraciones y el SQL

## 13. Continuación: comprobar lo aplicado

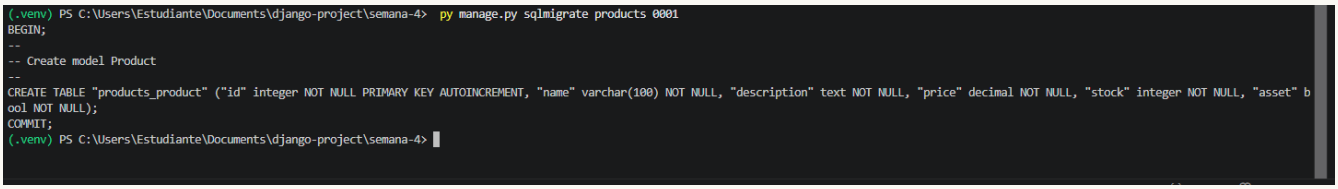
Al repetir **py manage.py migrate**, tu terminal muestra:

```
(.venv) PS C:\Users\Estudiante\Documents\django-project\semana-4>  py manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, products, sessions
Running migrations:
  No migrations to apply.
(.venv) PS C:\Users\Estudiante\Documents\django-project\semana-4>
```

## 3. Consultar el SQL de la migración

```
py manage.py sqlmigrate products 0001
```

Muestra el SQL correspondiente a la migración **0001** de **products**. Sirve para revisarlo; este comando no lo ejecuta ni vuelve a crear la tabla.




**Lo que ves:** la tabla contiene **id**, **name**, **description**, **price**, **stock** y **asset**, según el modelo que definiste.

**Para repetirlo:** cambiar models.py → makemigrations → migrate. Usa sqlmigrate cuando quieras consultar el SQL.

**Recuerda:** 0001 es la migración inicial de products; Django ordena las migraciones de todas las apps según sus dependencias.

## Nota importante

Si el modelo tenía un error y aplicamos la migración, ese error en la estructura puede quedar en la base de datos. Por eso es recomendable reparar el modelo y crear una **nueva migración** con **makemigrations**; después la aplicamos con **migrate**. Cambiar solamente models.py no actualiza la base de datos.

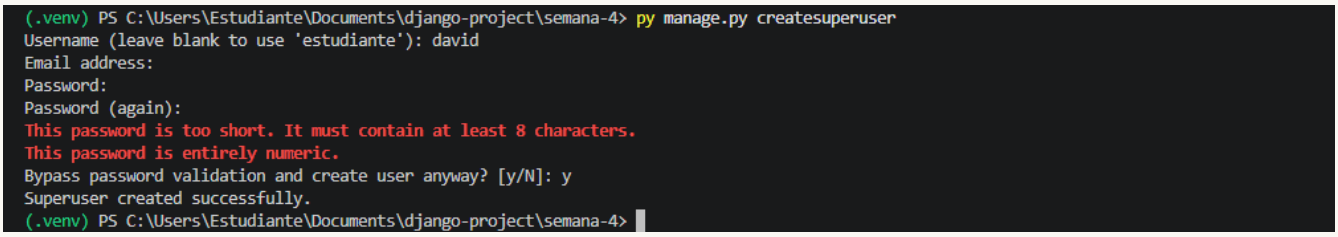
# Crear la cuenta de administración

## 14. Crear un superusuario

Desde la carpeta donde está **manage.py**, ejecuta:

```
py manage.py createsuperuser
```

Introduce el **nombre de usuario**, el **correo** (puedes dejarlo vacío en esta práctica) y la **contraseña dos veces**. La contraseña no se muestra al escribirla; es normal.




**Qué ocurrió:** Django avisó que la contraseña era demasiado corta y solo numérica. En tu práctica respondiste **y** para continuar y la cuenta se creó. Para repetirlo, usa una contraseña más larga y que no sea solo numérica.

**Recuerda bien la contraseña:** la necesitarás para iniciar sesión. Guarda tus credenciales en un lugar seguro; la contraseña no aparece en esta guía.

**¿Para qué sirve?** El superusuario permite acceder al sitio de administración de Django y gestionar los modelos registrados allí.

**Dos cosas distintas:** aquí creaste la cuenta; después registrarás el modelo **Product** en **products/admin.py** para que aparezca en el administrador.

Con el servidor iniciado, la página de administración está en:

```
http://localhost:8000/admin/
```

La cuenta está creada. En el paso 15 se registra Product y se accede al administrador.

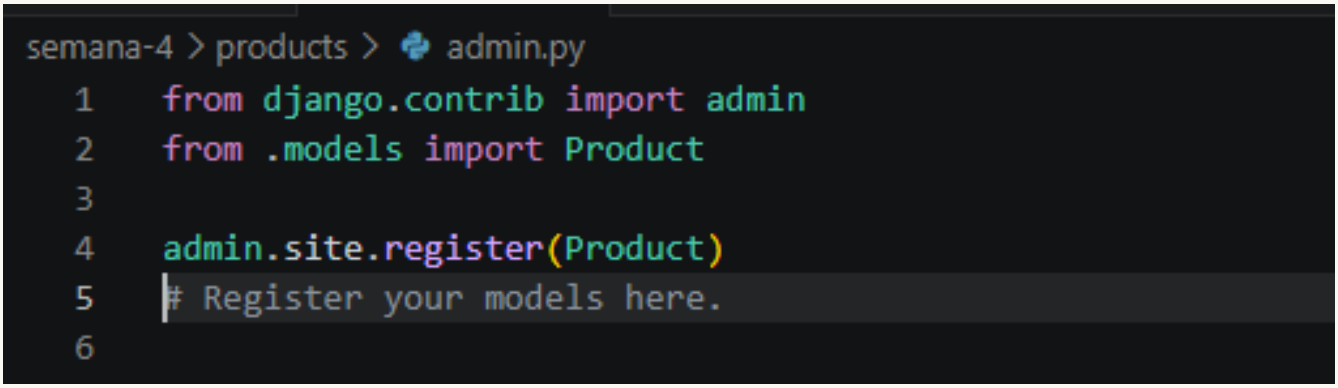
# Registrar Product y entrar al Admin

## 15. Administrar productos desde Django

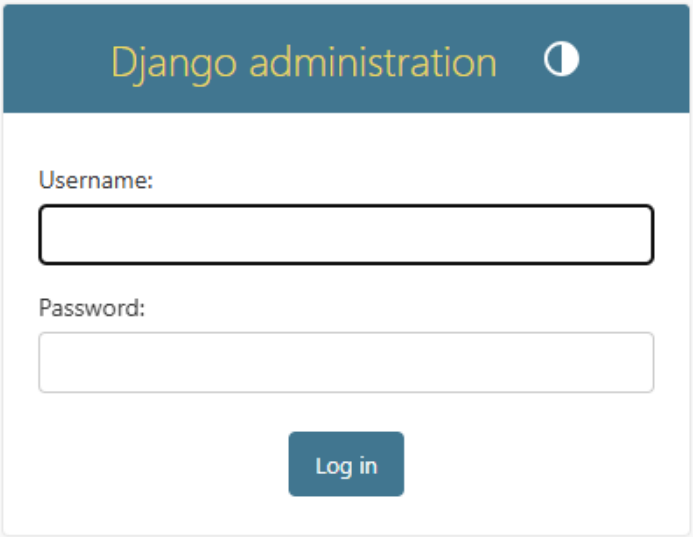
Abre **products/admin.py**, importa el modelo y regístralo:

```
from django.contrib import admin
from .models import Product

admin.site.register(Product)
```




Inicia el servidor con **py manage.py runserver** si está detenido. En el navegador, entra a **http://localhost:8000/admin/** e inicia sesión con tu usuario **david** y tu contraseña.

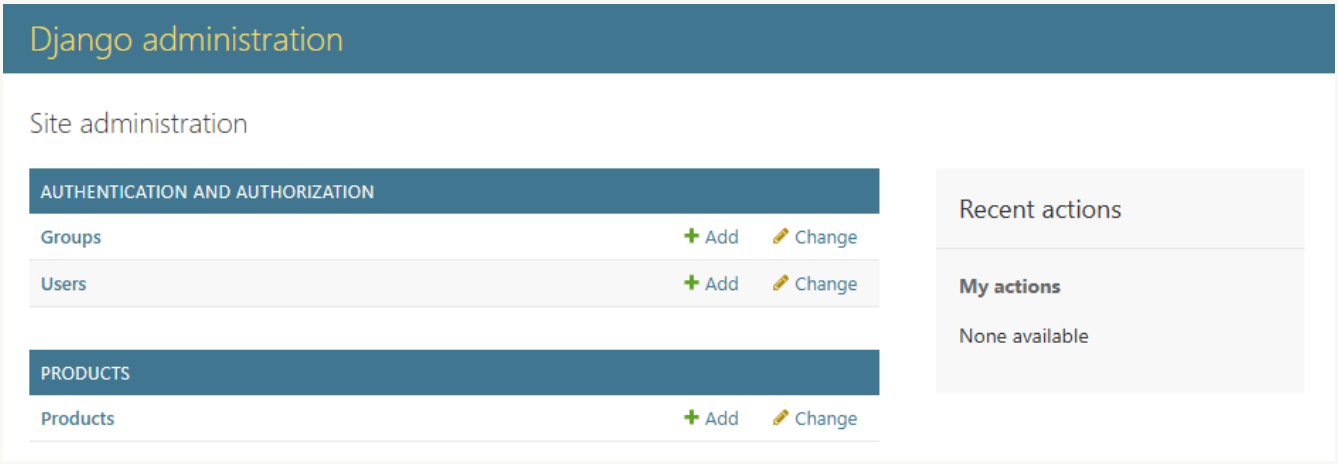



**Recuerda:** aquí registras el modelo, no al superusuario. Django genera la interfaz para administrar sus registros.

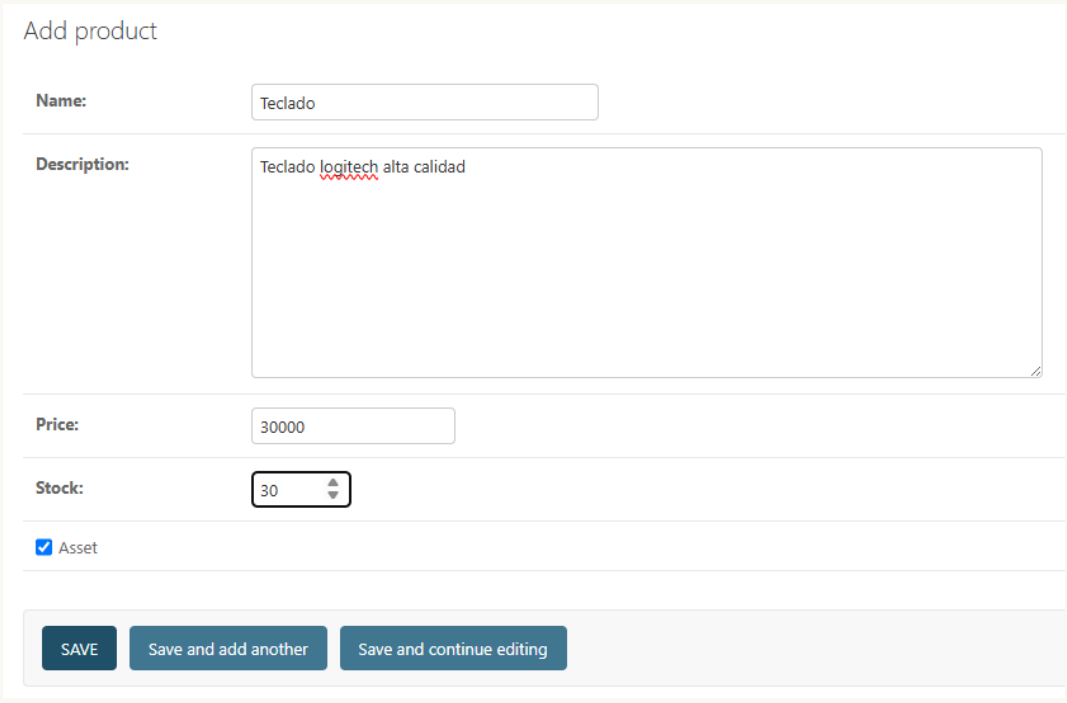
# Agregar productos desde el panel

## 15. Continuación: abrir el formulario

Al ingresar aparece el panel de administración. En **Products**, pulsa **Add** para abrir el formulario.




Completa **Name**, **Description**, **Price** y **Stock**. El formulario también incluye la casilla **Asset**, definida en tu modelo.




Pulsa **SAVE** para guardar el producto en la base de datos. La captura muestra el formulario completado; aún no se ha mostrado la confirmación de guardado.

## ¿Qué demuestra el Admin?

Crear un producto → **CREATE**.
Ver la lista → **READ**.
Editar un registro → **UPDATE**.

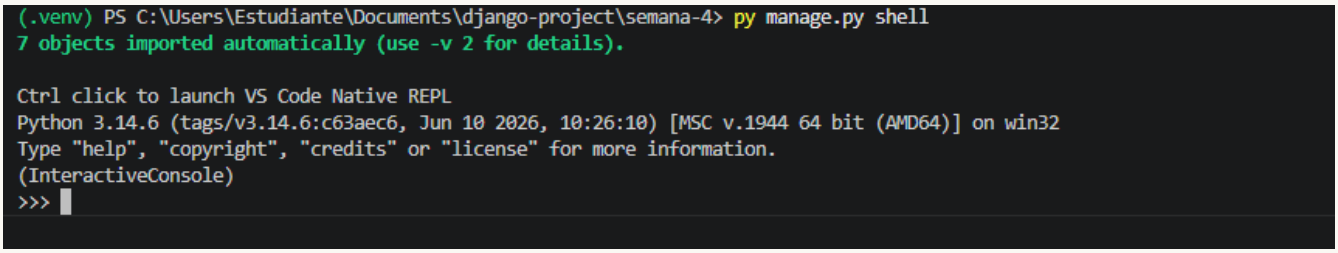
# Probar el ORM desde Python

## 16. Abrir Django Shell

Desde la terminal, en la carpeta donde está **manage.py**, ejecuta:

```
py manage.py shell
```

Abre una **consola interactiva de Python con el contexto del proyecto Django cargado**. Permite trabajar con los modelos y probar consultas del ORM directamente.




**Cómo reconocerla:** aparece **>>>**. A partir de ahí escribes código Python, incluidas operaciones de Django. En tu captura, la consola se abrió correctamente.

**La diferencia:** PowerShell es la terminal desde la que lanzaste el comando. Django Shell es la consola de Python que ahora se ejecuta dentro de ella.

**Objetivo de esta práctica:** usamos **Admin y Shell para hacer pruebas** y comprobar cómo funcionan los modelos y el ORM antes de conectarlos a las vistas. Ambos trabajan con la base de datos del proyecto: crear, editar o borrar registros modifica sus datos.

Para salir y regresar a PowerShell, escribe:

```
exit()
```

# Crear un producto con el ORM

## 17. CREATE desde Django Shell

Dentro de Django Shell, donde aparece **>>>**, importa el modelo y crea el producto:

```
from products.models import Product

Product.objects.create(
    name='Laptop',
    description='Equipo portatil',
    price=350000,
    stock=8,
    asset=True
)
```

```
>>> from products.models import Product
>>> Product.objects.create(name='Laptop', description = 'Equipo portatil', price=350000, stock=8, asset=True)
<Product: Laptop>
>>>
```

**Qué hace: create()** crea un objeto Product y lo guarda inmediatamente como un nuevo registro en la base de datos. Esta operación corresponde a **CREATE**.

**Qué es objects:** es el administrador del modelo, desde el que accedes a operaciones del ORM, como crear o consultar productos. Para editar o eliminar también puedes trabajar con los objetos recuperados.

**Resultado:** tu consola devuelve **<Product: Laptop>**. El nombre aparece gracias al método **__str__** definido en Product.

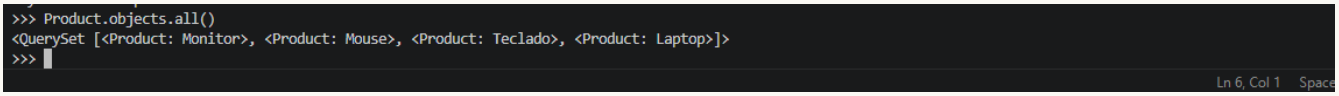
**Para practicar:** revisa la lista de Products en Admin; el nuevo registro debe aparecer allí. Si repites **create()**, crearás otro registro.

# Consultar los productos

## 17. Continuación del CRUD: READ

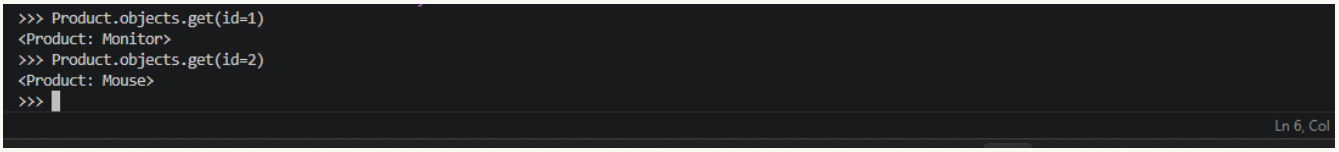
Acá básicamente es para ver todos los objetos creados hasta el momento:

```
Product.objects.all()
```




También tenemos **Product.objects.get** para ver específicamente por ID:

```
Product.objects.get(id=2)
```



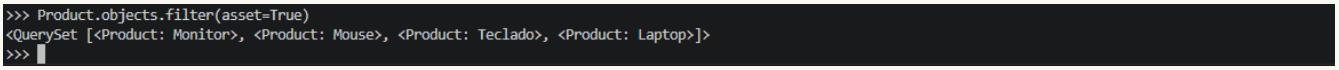

También se puede filtrar y buscar por stock. **stock__gt=10** significa stock mayor que 10:

```
Product.objects.filter(stock__gt=10)
```




También podemos filtrar por **asset**:

```
Product.objects.filter(asset=False)
Product.objects.filter(asset=True)
```



# Editar y eliminar un producto

## 17. Continuación del CRUD: UPDATE

Luego también podemos buscar un producto, cambiar su precio y stock, y guardar:

```
product = Product.objects.get(id=1)
product.price = 15000
product.stock = 2
product.save()
```

```
>>> Product.price = 15000
>>> Product.stock=2
>>> Product.save()
```

**Corrección mínima:** en la captura aparece **Product**, pero para modificar el objeto que obtuvimos usamos **product** en minúscula. **save()** guarda los cambios en la base de datos.

## DELETE

Seguidamente, para eliminar ese producto:

```
productDel = Product.objects.get(id=1)
productDel.delete()
```

Usamos el mismo nombre **productDel** en ambas líneas. **delete()** elimina el registro de la base de datos. Aún no se ha compartido el resultado de esta eliminación.

# Conectar la vista con los datos

## 18. Consultar productos y devolver JSON

Seguidamente conectaremos la vista con la base de datos. Nos dirigimos a **products/views.py**. Primero importamos **Product** desde models y también **JsonResponse**.

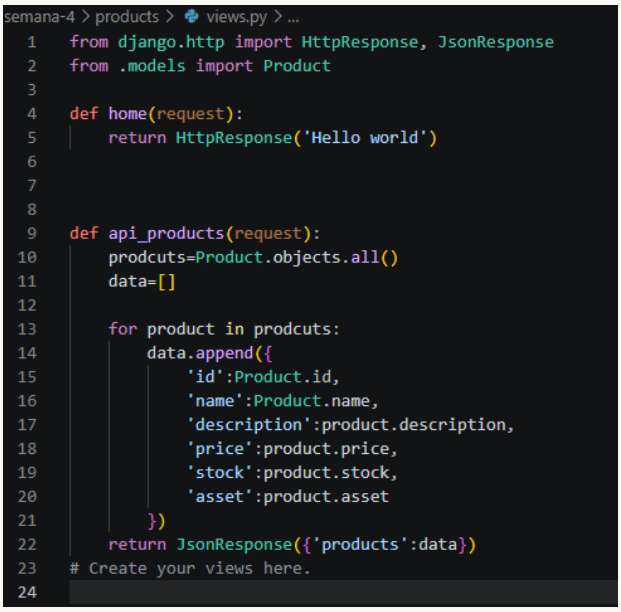
```
from django.http import HttpResponse, JsonResponse
from .models import Product

def api_products(request):
    products = Product.objects.all()
    data = []

    for product in products:
        data.append({
            'id': product.id,
            'name': product.name,
            'description': product.description,
            'price': product.price,
            'stock': product.stock,
            'asset': product.asset
        })

    return JsonResponse({'products': data})
```

Con **Product.objects.all()** obtenemos todos los productos guardados. **data = []** es la lista donde agregamos sus datos por medio del **for** y **data.append()**. Al final, **JsonResponse** devuelve esa lista dentro de **products**.




**Corrección mínima:** en la captura aparecen **Product.id** y **Product.name**. Deben ser **product.id** y **product.name**, usando el objeto que recorre el for. La vista **home** se conserva.

# Agregar la ruta y probar

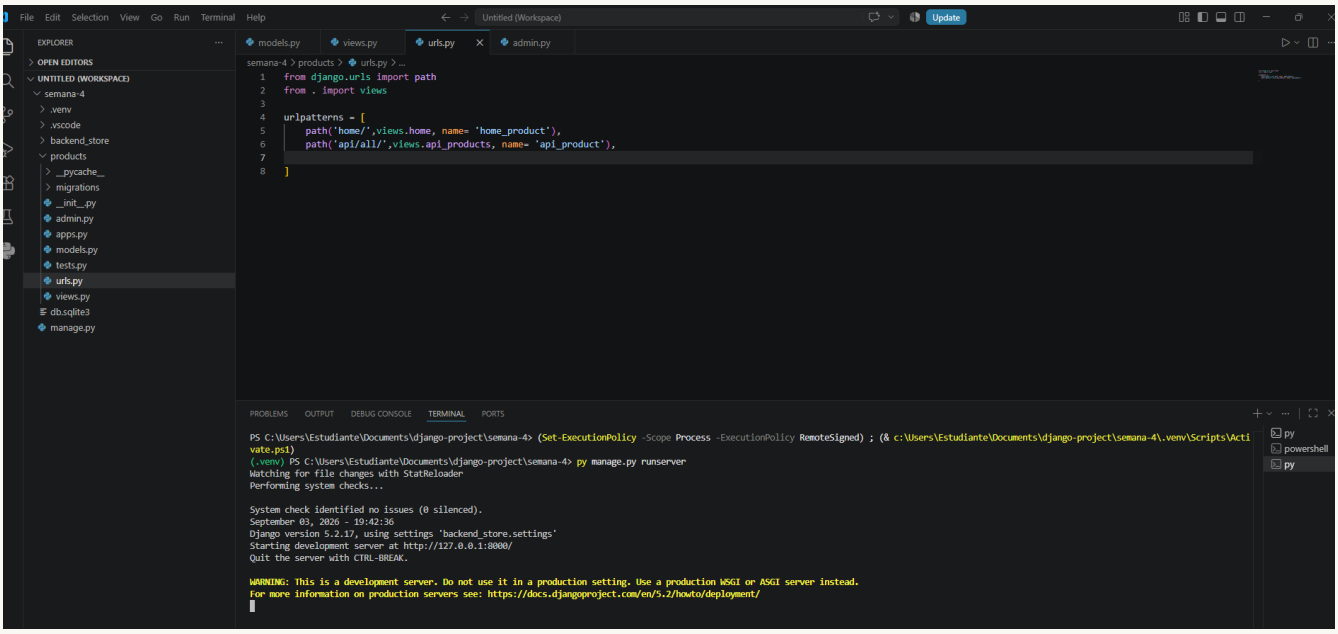
## 18. Continuación: conectar el endpoint

Una vez hecho lo de la vista en products, vamos a **products/urls.py** y agregamos el endpoint creado dentro de **urlpatterns**:

```
path('api/all/', views.api_products, name='api_product'),
```

Conservamos también la ruta **home/**. Seguidamente encendemos el servidor para probar:

```
py manage.py runserver
```




Abrimos en el navegador:

```
http://localhost:8000/products/api/all/
```

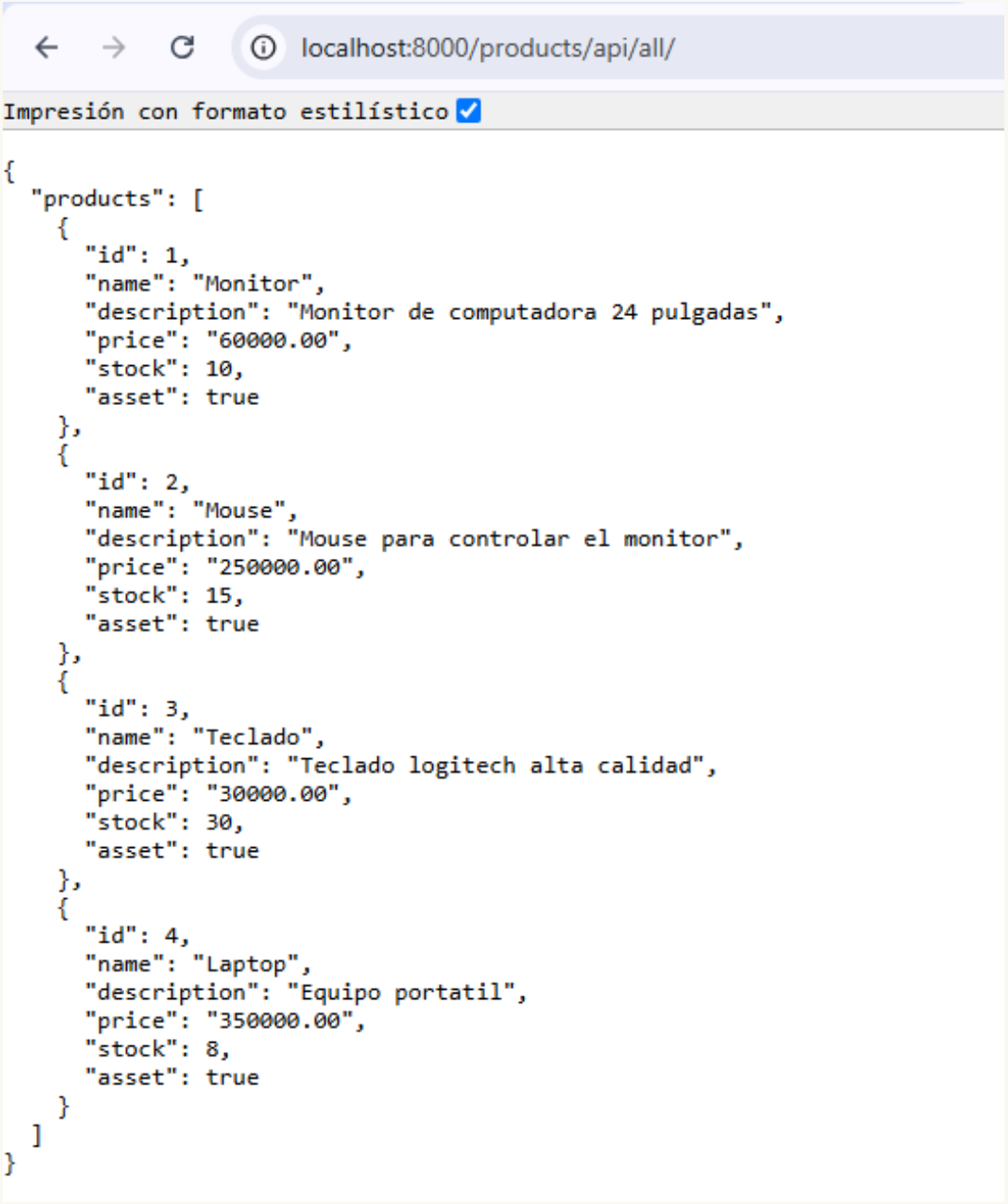
El prefijo **products/** del proyecto se une con **api/all/** de la app. Esta dirección debe devolver el JSON con los productos guardados.

La captura confirma que el servidor inició sin errores en las comprobaciones. La respuesta en el navegador se muestra a continuación.

# Resultado de la consulta

### 18. Continuación: respuesta en el navegador

Y seguidamente se ve de esta manera: al abrir **http://localhost:8000/products/api/all/**, aparecen los productos en formato JSON.




Dentro de **products** se muestran los cuatro productos: **Monitor, Mouse, Teclado y Laptop**, con sus respectivos datos. Así comprobamos que la vista consulta la base de datos y devuelve la respuesta.

# Otra forma: usar values()

## 18. Continuación: convertir la consulta en una lista

**all()** devuelve los objetos completos del modelo, por eso en el ejemplo anterior sacamos sus datos uno por uno. Aquí utilizamos **values()** para obtener los campos elegidos como diccionarios, convertirlos en una lista y retornarlos.

```
def api_products_all(request):
    products = Product.objects.values(
        'id', 'name', 'description', 'price', 'stock', 'asset'
    )
    return JsonResponse({'products': list(products)})
```

```
8
9   def api_products(request):
10      prodcuts=Product.objects.all()
11      data=[]
12
13      for product in prodcuts:
14          data.append({
15              'id':product.id,
16              'name':product.name,
17              'description':product.description,
18              'price':product.price,
19              'stock':product.stock,
20              'asset':product.asset
21          })
22      return JsonResponse({'products':data})
23
24
25  def api_products_all(request):
26      products=Product.objects.values(
27          'id','name','description','price','stock','asset'
28      )
29      return JsonResponse({'products':list(products)})
30
```

**Esta es una manera más corta de realizarlo. list(products)** convierte el resultado de la consulta en una lista para enviarla en el JSON. Así no necesitamos escribir el for del ejemplo anterior.

**Recordar:** ir a **products/urls.py** y agregar el **path** para **views.api_products_all**. Por ejemplo, usando una ruta distinta a la que ya existe:

```
path('api/values/', views.api_products_all,
     name='api_products_values'),
```

Con esa ruta de ejemplo, se prueba en **http://localhost:8000/products/api/values/**.